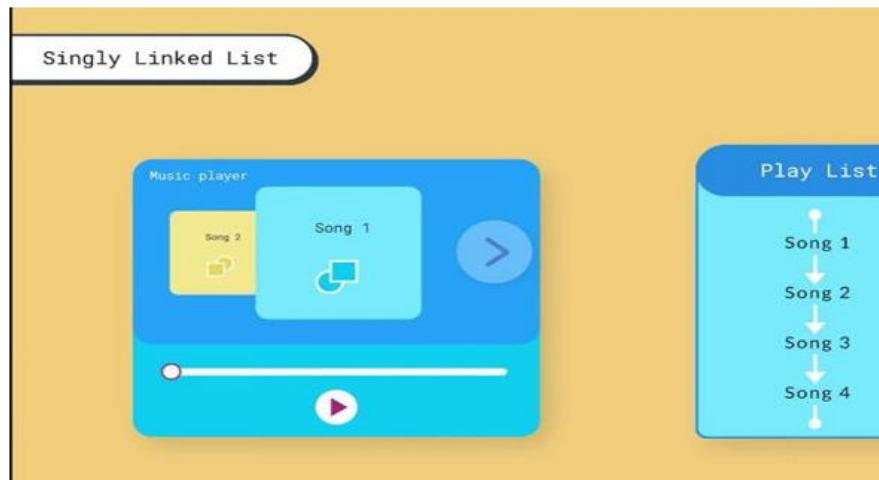




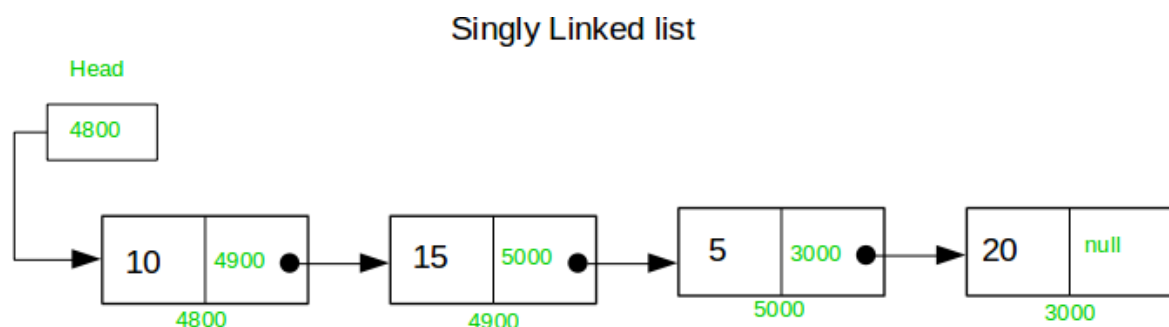
# Singly Linked List in Python

## Introduction

A **singly linked list** is a linear data structure where each element (called a **node**) contains two parts:

1. **Data:** The actual value stored in the node
2. **Next:** A reference to the next node in the list (or `None` if it's the last node)

Unlike arrays, linked lists do not store elements in contiguous memory locations. Instead, each node is dynamically allocated and linked to the next node through references.



## Node Structure in Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

# Why Python Doesn't Need Double Pointers

Unlike C/C++, Python doesn't require double pointers for modifying the head of a linked list because:

## 1. Object References in Python

- In Python, variables are references to objects
- When you modify `self.head` inside a method, you're modifying the instance attribute directly
- The `self` parameter provides access to the instance, allowing modification of its attributes

## 2. Automatic Memory Management

- Python handles memory allocation and deallocation automatically
- No need to manually manage pointers or memory addresses
- Garbage collection handles cleanup of unreferenced nodes

## 3. Method Binding

- Methods are bound to the instance, so `self.head` always refers to the same attribute
- Changes to `self.head` are persistent across method calls

## 4. Example Comparison

### C++ (requires double pointer):

```
void insertAtBegin(Node** head, int data) {
    Node* newNode = new Node(data);
    newNode->next = *head;
    *head = newNode; // Modifying the original head pointer
}
```

### Python (direct attribute modification):

```
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node # Directly modifying the instance attribute
```

## Advantages of Singly Linked Lists

1. **Dynamic Size:** Can grow or shrink during runtime
2. **Memory Efficient:** Only allocates memory as needed
3. **Easy Insertion/Deletion:** Efficient insertion and deletion at the beginning
4. **No Memory Waste:** No pre-allocated unused memory

## Disadvantages of Singly Linked Lists

1. **Sequential Access:** Cannot directly access elements by index
2. **Extra Memory:** Requires additional memory for storing pointers
3. **No Backward Traversal:** Can only traverse in one direction

## Time Complexity

| Operation               | Time Complexity |
|-------------------------|-----------------|
| Insertion at beginning  | $O(1)$          |
| Insertion at end        | $O(n)$          |
| Insertion at position   | $O(n)$          |
| Deletion from beginning | $O(1)$          |
| Deletion from end       | $O(n)$          |
| Deletion by value       | $O(n)$          |
| Search                  | $O(n)$          |
| Traversal               | $O(n)$          |

## Space Complexity

- **Space Complexity:**
- 
- **Extra Space:**  $O(1)$  for each node (one pointer per node)

## Complete Implementation

**#sngle linked list (add the node at begin)**

class Node:

```
def __init__(self,data):
```

```
    self.data = data
```

```
    self.ref = None
```

class Linkelist:

```
def __init__(self):
```

```
    self.head = None
```

```
def add_begin(self,data):
```

```
    newnode = Node(data)
```

```
newnode.ref = self.head
```

```
self.head = newnode
```

```
def display(self):
```

```
    if self.head == None:
```

```
        print("Linked list is empty")
```

```
    else:
```

```
        n = self.head
```

```
        while n != None:
```

```
            print(n.data,"-->",end=" ")
```

```
            n = n.ref
```

```
        print("Null")
```

```
l = Linkedlist()
```

```
l.add_begin(10)
```

```
l.add_begin(20)
```

```
l.add_begin(30)
```

```
l.display()
```

### **Output:**

```
30 --> 20 --> 10 --> Null
```

### **#single linked list (add the node at end)**

```
class Node:
```

```
    def __init__(self,data):
```

```
        self.data = data
```

```
        self.ref = None
```

```
class Linkedlist:
```

```
    def __init__(self):
```

```
        self.head = None
```

```

def add_end(self,data):
    newnode = Node(data)
    if self.head is None:
        self.head = newnode
    else:
        n = self.head
        while n.ref != None:
            n = n.ref
        n.ref = newnode

def display(self):
    if self.head == None:
        print("Linked list is empty")
    else:
        n = self.head
        while n != None:
            print(n.data,"-->",end=" ")
            n = n.ref
        print("Null")

```

```
l = Linkedlist()
```

```
l.add_end(10)
```

```
l.add_end(20)
```

```
l.add_end(30)
```

```
l.display()
```

**output:**

```
10 --> 20 --> 30 --> Null
```

**#sigle linked list add at specific position**

```
class Node:
```

```

def __init__(self,data):
    self.data = data
    self.ref = None
class Linkedlist:
    def __init__(self):
        self.head = None

    def add_begin(self,data):
        newnode = Node(data)
        newnode.ref = self.head
        self.head = newnode

    def addatpos(self,pos,data):
        newnode = Node(data)
        if pos == 1:
            newnode.ref = self.head
            self.head = newnode
            return
        n = self.head
        i = 1
        while n != None and i < pos - 1:
            n = n.ref
            i += 1
        if n is None:
            print("Position out of range")
        else:
            newnode.ref = n.ref
            n.ref = newnode

    def display(self):
        if self.head == None:

```

```

        print("Linked list is empty")
    else:
        n = self.head
        while n != None:
            print(n.data,"-->",end=" ")
            n = n.ref
        print("Null")

```

```

l = Linkedlist()
l.add_begin(10)
l.add_begin(20)
l.add_begin(30)
l.display()
l.addatpos(3,50)
l.display()

```

#### **output:**

```

30 --> 20 --> 10 --> Null
30 --> 20 --> 50 --> 10 --> Null

```

#### **#sigle linked list delete at begin**

```

class Node:
    def __init__(self,data):
        self.data = data
        self.ref = None
class Linkedlist:
    def __init__(self):
        self.head = None

    def add_begin(self,data):
        newnode = Node(data)
        newnode.ref = self.head

```

```

        self.head = newnode

def delatbegin(self):
    if self.head == None:
        print("Linked list is Empty")
    else:
        self.head = self.head.ref

def display(self):
    if self.head == None:
        print("Linked list is empty")
    else:
        n = self.head
        while n != None:
            print(n.data,"-->",end=" ")
            n = n.ref
        print("Null")

l = Linkedlist()
l.add_begin(10)
l.add_begin(20)
l.add_begin(30)
l.display()
l.delatbegin()
l.display()

```

**output:**

```

30 --> 20 --> 10 --> Null
20 --> 10 --> Null

```

**#sigle linked list delete at end**

```

class Node:
    def __init__(self,data):

```

```

        self.data = data

        self.ref = None
class Linkelist:
    def __init__(self):
        self.head = None

    def add_begin(self,data):
        newnode = Node(data)
        newnode.ref = self.head
        self.head = newnode

    def delatend(self):
        if self.head == None:
            print("Linked list is Empty")
        elif self.head.ref == None:
            self.head = None
        else:
            n = self.head
            while n.ref.ref != None:
                n = n.ref
            n.ref = None

    def display(self):
        if self.head == None:
            print("Linked list is empty")
        else:
            n = self.head
            while n != None:
                print(n.data,"-->",end=" ")
                n = n.ref
            print("Null")

```

```

l = Linklist()
l.add_begin(10)
l.add_begin(20)
l.add_begin(30)
l.display()
l.delatend()
l.display()

```

**output:**

```

30 --> 20 --> 10 --> Null
30 --> 20 --> Null

```

**#sigle linked list delete at specific position**

```

class Node:
    def __init__(self,data):
        self.data = data
        self.ref = None
class Linklist:
    def __init__(self):
        self.head = None

    def add_begin(self,data):
        newnode = Node(data)
        newnode.ref = self.head
        self.head = newnode

    def delatpos(self,x):
        if self.head is None:
            print("Linked list is empty")
            return
        if self.head.data == x:

```

```

        self.head = self.head.ref

    return

n = self.head
while n.ref != None:
    if n.ref.data == x:
        n.ref = n.ref.ref
        return
    n = n.ref
print("Vlue not found")

```

```

def display(self):
    if self.head == None:
        print("Linked list is empty")
    else:
        n = self.head
        while n != None:
            print(n.data,"-->",end=" ")
            n = n.ref
        print("Null")

```

```

l = Linkedlist()
l.add_begin(10)
l.add_begin(20)
l.add_begin(30)
l.display()
l.delatpos(10)
l.display()

```

**output:**

```

30 --> 20 --> 10 --> Null
30 --> 20 --> Null

```

### **#sigle linked list searching techniques**

class Node:

```
def __init__(self,data):
```

```
    self.data = data
```

```
    self.ref = None
```

class Linkelist:

```
def __init__(self):
```

```
    self.head = None
```

```
def add_begin(self,data):
```

```
    newnode = Node(data)
```

```
    newnode.ref = self.head
```

```
    self.head = newnode
```

```
def search (self,x):
```

```
    n = self.head
```

```
    pos = 1
```

```
    while n != None:
```

```
        if n.data == x:
```

```
            print(f"Element {x} fount at position {pos}")
```

```
            return
```

```
        n = n.ref
```

```
        pos += 1
```

```
    print(f"Element {x} not found")
```

```
def display(self):
```

```
    if self.head == None:
```

```
        print("Linked list is empty")
```

```
    else:
```

```
        n = self.head
```

```
        while n != None:
```

```
        print(n.data,"-->",end=" ")
        n = n.ref
    print("Null")
```

```
l = Linkedlist()
l.add_begin(10)
l.add_begin(20)
l.add_begin(30)
l.display()
l.search(30)
```

**output:**

```
30 --> 20 --> 10 --> Null
Element 30 found at position 1
```

**#single linked list Reverse all nodes techniques**

```
class Node:
```

```
    def __init__(self,data):
        self.data = data
        self.ref = None
```

```
class Linkedlist:
```

```
    def __init__(self):
        self.head = None
```

```
    def add_begin(self,data):
        newnode = Node(data)
        newnode.ref = self.head
        self.head = newnode
```

```
    def reverse(self):
        prev = None
        curt = self.head
```

```
while curt != None:
    nextnode = curt.ref
    curt.ref = prev
    prev = curt
    curt = nextnode
self.head = prev
```

```
def display(self):
    if self.head == None:
        print("Linked list is empty")
    else:
        n = self.head
        while n != None:
            print(n.data,"-->",end=" ")
            n = n.ref
        print("Null")
```

```
l = Linklist()
l.add_begin(10)
l.add_begin(20)
l.add_begin(30)
l.display()
l.reverse()
l.display()
```

**output:**

```
30 --> 20 --> 10 --> Null
10 --> 20 --> 30 --> Null
```